Security Tokens & Modern Web Architecture

OAuth 2.0 · JWT · Session Management · LLM Tokenisation

A Comprehensive Technical Reference Guide · 2025 Edition

PART 1 — Asset Management in Software Systems

What is a Digital Asset?

A digital asset is any resource with measurable value within a software ecosystem — encompassing data, credentials, keys, media, and intellectual property. Proper asset governance is a foundational pillar of modern security architecture.

- **Digital assets:** API keys, tokens, certificates, user data, and media files
- **Infrastructure assets:** Servers, databases, and cloud storage buckets
- **Classification levels:** Public → Internal → Confidential → Restricted
- **Asset lifecycle:** Inventory + ownership + access control + rotation/expiry

Asset Classification Framework

Level	Examples	Access Scope
PUBLIC	Marketing images, public docs, open APIs	Anyone
INTERNAL	Employee directories, internal APIs	Staff only
CONFIDENTIAL	PII, payment info, API secrets	Role-based
RESTRICTED	Private keys, root credentials, HSMs	Break-glass

Secrets Management Best Practices

Secrets — database passwords, API keys, tokens — must never be hardcoded in source code. Use a dedicated secrets manager and enforce rotation policies.

- Never commit .env files or secrets to version control
- Recommended vaults: **AWS Secrets Manager**, **HashiCorp Vault**, **Azure Key Vault**
- Rotate secrets on a schedule; revoke immediately on team member departure
- Scan repositories for leaks using **GitGuardian**, **truffleHog**, or **git-secrets**

```
# Bad — hardcoded secret (never do this) DB_PASSWORD = 'super_secret_123' # Good — loaded from environment or vault at runtime import os DB_PASSWORD = os.getenv('DB_PASSWORD') # or fetched dynamically from AWS SSM / HashiCorp Vault
```

PART 2 — OAuth 2.0 Deep Dive

What is OAuth 2.0?

OAuth 2.0 is an open authorisation framework that enables a third-party application to access a user's resources on their behalf — without ever exposing their credentials. It issues scoped, time-limited access tokens instead of passwords.

- **Delegates access:** The user never shares their password with the client application
- **4 Core Roles:** Resource Owner · Client · Authorisation Server · Resource Server
- **Token-based:** Short-lived access tokens scoped to specific permissions
- Widely adopted by **Google**, **GitHub**, **Apple**, and **Microsoft** for social login

> ■ *OAuth 2.0 handles AUTHORISATION (what you can do), not authentication (who you are). OpenID Connect (OIDC) adds an identity layer on top of OAuth 2.0.*

Grant Types

Grant Type	Use Case	Security Level
Authorization Code + PKCE	Web & mobile apps	Highest
Client Credentials	Machine-to-machine (M2M)	High
Device Code	Smart TVs, CLIs, IoT	Medium
Implicit (deprecated)	Legacy SPAs — avoid!	Low

Access Token vs Refresh Token

- **Access token:** Short-lived (15 min – 1 hr), included in every API request header
- **Refresh token:** Long-lived (days–weeks), stored securely, never sent to APIs
- **Token rotation:** Issue a new refresh token on each use; invalidate the previous one
- Store refresh tokens in HTTP-only cookies — never in localStorage (XSS risk)

```
# Typical token lifetimes access_token → expires_in: 3600 (1 hour) refresh_token → expires_in: 2592000 (30 days) # Refreshing an access token POST /oauth/token grant_type=refresh_token refresh_token=<stored_refresh_token>
```

PART 3 — JSON Web Tokens (JWT)

JWT Structure & Claims

A JSON Web Token is a compact, URL-safe, self-contained token that encodes claims as a JSON object and is cryptographically signed. It consists of three Base64url-encoded parts separated by dots: Header · Payload · Signature.

- **Stateless:** The server validates the token without any database lookup
- **Signing algorithms:** HS256 (symmetric) or RS256/ES256 (asymmetric — preferred)
- **Registered claims:** iss, sub, aud, exp, iat, nbf, jti
- **Private claims:** Custom fields like roles, tenant_id, permissions

```
{ "iss": "https://auth.myapp.com", // Issuer "sub": "user_42", // Subject (User ID) "aud": "https://api.myapp.com", // Audience "exp": 1750000000, // Expiry (Unix timestamp) "iat": 1749996400, // Issued-at "roles": ["admin", "editor"], // Custom claim "jti": "unique-token-id-abc123" // JWT unique ID }
```

■ *JWTs are Base64url-encoded, NOT encrypted. Anyone can decode the payload — never embed passwords or sensitive PII unless using JWE (JSON Web Encryption).*

JWT Security Best Practices

- Always **reject the "none" algorithm** — attackers can forge unsigned tokens
- Prefer **RS256 (asymmetric)** over HS256 when multiple services verify tokens
- Keep access JWTs **short-lived (≤ 15–60 min)**;use refresh tokens for longevity
- Store in memory or HTTP-only cookies — NOT localStorage (XSS vulnerability)
- Rotate signing keys periodically; use the key ID (kid) header to manage key sets

```
// Node.js — always specify allowed algorithms explicitly import jwt from 'jsonwebtoken'; // Correct: explicit algorithm check jwt.verify(token, PUBLIC_KEY, { algorithms: ['RS256'], audience: 'https://api.myapp.com', issuer: 'https://auth.myapp.com', }); // Vulnerable: no algorithm specified (accepts 'none' in some libraries!) jwt.verify(token, key); // NEVER do this
```

PART 4 — State, Object & Session Management

Session Management

A session maintains user context across multiple HTTP requests, since HTTP itself is inherently stateless. Session security is a critical defence layer against fixation, hijacking, and CSRF attacks.

- **Server-side sessions:** Session ID in cookie; session data stored in Redis/DB — fully revocable
- **Client-side sessions:** JWT in a cookie — stateless, but harder to revoke mid-flight
- Always set **Secure**, **HttpOnly**, **SameSite=Strict** cookie attributes
- Implement both idle timeout and absolute session expiry
- Regenerate the session ID immediately after successful login to prevent session fixation

```
# Secure session cookie — recommended attributes Set-Cookie: sessionId=abc123; HttpOnly; # No JS access → blocks XSS theft Secure; # HTTPS only SameSite=Strict; # No cross-site requests → prevents CSRF Max-Age=3600; # 1-hour expiry Path=/;
```

State Storage Reference

Use Case	Recommended Storage
Short-lived / ephemeral	In-memory (Redis, Memcached)
User session data	Redis session store
Long-term persistence	PostgreSQL, MongoDB

Client-side (safe)	HTTP-only cookie or memory
Client-side (avoid)	localStorage, sessionStorage

PART 5 — Token Identifiers & Unique ID Standards

Identifier Formats Compared

Format	Length	Sortable?	Best For
UUID v4	36 chars	No	General-purpose unique IDs
UUID v7	36 chars	Yes (time-prefix)	DB primary keys
ULID	26 chars (base32)	Yes	Distributed systems, logs
NanoID	21 chars (default)	No	URLs, public-facing IDs
API Key	Variable	N/A	Static app authentication

```
UUID v4: 550e8400-e29b-41d4-a716-446655440000 (random) UUID v7:
018fce17-b8a1-7f5e-9c2d-3a4b5c6d7e8f (time-sortable) ULID: 01ARZ3NDEKTSV4RRFFQ69G5FAV
(sortable, compact) NanoID: V1StGXR8_Z5jdHi6B-myT (URL-safe, tiny)
```

■ *Always use UUIDs or ULIDs for public-facing identifiers. Exposing sequential integer IDs leaks record counts and enables enumeration attacks.*

PART 6 — LLM Tokenisation & Model Comparison

How Large Language Models Tokenise Text

LLMs do not process raw characters or whole words. Instead, they operate on tokens — subword units produced by a trained tokeniser. Token count directly affects API cost, context window usage, and inference speed.

- Rule of thumb: 1 token ≈ 0.75 English words, or approximately 4 characters
- Punctuation, whitespace, and letter casing all influence token boundaries
- Emojis and special characters are expensive — typically 2–6 tokens each
- **Byte Pair Encoding (BPE)** is the dominant algorithm used by GPT, LLaMA, and most modern models

```
# Tokenisation of "unhappiness" across strategies: Character tokens : u n h a p p i n e s s
→ 11 tokens Word token : unhappiness → 1 token BPE subword : un + happi + ness → 3 tokens
(typical LLM output) # Same text, different models: "def calculate_total(items):" GPT-4o
(cl100k): [def] [ calculate] [_total] [(] [items] [):] → 6 tokens Mistral (32k vocab): → 8
tokens
```

Major LLM Tokeniser Comparison

Model	Tokeniser	Vocab Size	Context Window
GPT-4o (OpenAI)	cl100k_base (tiktoken BPE)	~100,277	128K tokens
GPT-4.1 (OpenAI)	o200k_base (tiktoken BPE)	~200,019	1M tokens

Claude 3.5/4 (Anthropic)	Custom BPE (sentencepiece)	~100K+	200K tokens
Gemini 1.5/2 (Google)	SentencePiece Unicode BPE	~256K	1M – 2M tokens
LLaMA 3.x (Meta)	tiktoken cl100k_base	~128,256	128K tokens
Mistral / Mixtral	SentencePiece BPE	~32,000	32K – 128K
Grok 3 (xAI)	Custom BPE	~100K+	128K tokens
Command R+ (Cohere)	Custom BPE (sentencepiece)	~256K	128K tokens

Context Window Sizes

Model	Context Window	Approx. Word Equivalent
GPT-3.5	16K tokens	~12,000 words
GPT-4o	128K tokens	~96,000 words
Claude 3.5 / 4	200K tokens	~150,000 words
GPT-4.1	1M tokens	~750,000 words
Gemini 1.5 Pro	2M tokens	~1,500,000 words

■ *A 1M token context can accommodate roughly 800,000 words — equivalent to about 10 full novels. Fewer tokens = lower API cost and more room for reasoning within the window.*